

FORMATION EMBARQUÉE

TP3 — Seance 2

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 3 — Fiche de séance 2 : programme structuré en blocs (3 h)

En-tête pédagogique

Objectifs	Créer un projet TIA + CPU + tags ; structurer en OB/FC/FB/DB ; traduire le GRAFCET en SCL ; centraliser les sorties
Prérequis	Séance 1 : GRAFCET et liste d'E/S validés
Outils	TIA Portal (essai) + PLCSIM
Livrable	Projet qui compile sans warning, table de visualisation prête

Déroulé minuté

0:00-0:20 — Projet, CPU, tags

1. Créer un projet → « Ajouter un appareil » → CPU 1214C DC/DC/DC.
2. **Table des variables API** : saisir toute la liste d'E/S de la séance 1 (noms symboliques !). Ne jamais écrire `%I0.0` dans le code — toujours `"S1_marche"`.
3. Ajouter les mémentos et un DB : `"DB_IHM"` (global, DB optimisé) avec `marche_demandee`, `temps_remplissage_ms` (Int), `taille_lot` (Int), `compteur` (Int), `etape` (Int).

0:20-0:40 — L'architecture imposée

Bloc	Type	Rôle
<code>Main [OB1]</code>	OB	appelle les blocs dans l'ordre, RIEN d'autre
<code>FC_Modes</code>	FC	calcule <code>auto_actif</code> / <code>manu_actif</code> , demande marche (auto-maintien)
<code>FB_Defaults</code>	FB	AU, bourrage (TON 15 s), mémorisation, S6 → <code>aucun_default</code>
<code>FB_Sequence</code>	FB	le GRAFCET en SCL (<code>CASE #etape</code>)
<code>FC_Sorties</code>	FC	seul bloc qui écrit les %Q

Pourquoi ce découpage ? Deux règles d'or de la maintenabilité : 1. **Un seul bloc écrit les sorties.** Quand un technicien cherchera « pourquoi KM1 ne tourne pas », il ouvrira UN réseau. Sorties éparpillées = cauchemar (et double écriture = bug : la dernière gagne). 2. **La sécurité est en aval, en dernier.** Quelle que soit la fantaisie de la séquence, `FC_Sorties` coupe tout sur défaut.

0:40-1:00 — L'ordre d'appel dans OB1

```
// OB1 – orchestration pure
"FC_Modes"();
"FB_Defaults"(DB_Defaults);           // FB → son DB d'instance
"FB_Sequence"(DB_Sequence);
"FC_Sorties"();                       // TOUJOURS en dernier
```

L'ordre compte : les modes et défauts sont calculés AVANT la séquence (qui les lit), et les sorties APRÈS tout le monde.

1:00-2:15 — Coder FB_Sequence en SCL

Interface du FB : Static `etape : Int`, `t_rempli : TON_TIME`, `t_stab : TON_TIME`. Corps :

```
CASE #etape OF
  0: // Attente
    IF "DB_IHM".marche_demandee AND #auto AND #aucun_defaut THEN
      #etape := 10;
    END_IF;

  10: // Convoyage
    IF "S4_bouteille" THEN #etape := 20; END_IF;

  20: // Remplissage
    #t_rempli(IN := TRUE,
              PT := DINT_TO_TIME(INT_TO_DINT("DB_IHM".temps_remplissage_ms)));
    IF #t_rempli.Q THEN
      #t_rempli(IN := FALSE);           // réarmer en quittant l'étape
      #etape := 30;
    END_IF;

  30: // Stabilisation + incrément lot
    #t_stab(IN := TRUE, PT := T#1s);
    IF #t_stab.Q THEN
      #t_stab(IN := FALSE);
      "DB_IHM".compteur := "DB_IHM".compteur + 1; // au franchissement
      IF "DB_IHM".compteur >= "DB_IHM".taille_lot THEN
        #etape := 40;
      ELSE
        #etape := 10;
      END_IF;
    END_IF;

  40: // Lot terminé
    IF "S6_acquit" THEN
      "DB_IHM".compteur := 0;
      #etape := 0;
    END_IF;
END_CASE;
```

Note : les numéros d'étape = ceux du GRAFCET dessiné. Le dessin est le document de référence ; le code n'est que sa traduction. Mets le GRAFCET en commentaire en tête du bloc.

2:15-2:45 — FC_Sorties : la centralisation

```
// SEUL bloc qui écrit les %Q. La sécurité est intégrée ici, en dernier.
"KM1_convoyeur" := ("DB_Sequence".etape = 10 OR "DB_Sequence".etape = 30
                    OR #cmd_manu_convoyeur)
                    AND "FB_Defaults".aucun_defaut;

"YV1_vanne" := ("DB_Sequence".etape = 20 OR #cmd_manu_vanne)
               AND "FB_Defaults".aucun_defaut;

"H1_defaut" := NOT "FB_Defaults".aucun_defaut;
"H2_lot"    := ("DB_Sequence".etape = 40);
```

Observe : même si la séquence bugue et met KM1 à 1 au mauvais moment, `AND aucun_defaut` le coupe. C'est la **défense en profondeur**.

2:45-3:00 — Compilation et table de visu

Compile (Ctrl+B) : **zéro erreur, zéro avertissement** (un warning « variable non initialisée » cache souvent un vrai bug). Crée une table de visualisation `Visu_Cycle` avec : `etape`, toutes les %I, toutes les %Q, `compteur`, les `.Q` des tempos.

✅ **Point de contrôle** : compilation propre + table prête. Commit du projet (export ou dossier archivé + note dans ton journal Git perso).

Erreurs fréquentes

Symptôme	Cause	Remède
Compteur s'incrémente en boucle	incrément dans une action continue, pas au franchissement	mettre l'incrément dans le <code>IF tempo.Q</code>
Tempo qui ne redémarre jamais	<code>IN</code> jamais remis à FALSE	réarmer en quittant l'étape
KM1 reste collé sur défaut	sortie écrite dans plusieurs blocs	centraliser dans FC_Sorties
Warning « accès non initialisé »	variable lue avant écriture	initialiser dans OB100 ou à la déclaration
Séquence bloquée en étape 20	<code>PT</code> mal converti (Int → Time)	<code>DINT_TO_TIME</code> sur des ms

Travail à la maison (45 min)

Code `FB_Defaults` complet : l'AU (mémorisation sur $S3=0$, acquit sur $S6+S3=1$), le bourrage (TON 15 s armé par `S4 AND NOT(etape IN {20,30})`), et la sortie `aucun_defaut`. Teste mentalement : que se passe-t-il si S6 est appuyé alors que l'AU est toujours enfoncé ? (*rien : la cause doit d'abord disparaître*).

➡ Fiche suivante : **Séance 3 — Mise au point PLCSIM**